

SISTEMAS DE COMPUTADORES – 2024/2025
Exame Época Especial

Versão A

Apenas autorizada a consulta da folha oficial
Duração: 2h

Nome: _____ Nº _____

Grupo I
(8/20 valores)

NOTAS:

- 1. Em todas as questões deverá assinalar apenas uma resposta.**
- 2. Se a resposta assinalada for incorreta sofrerá uma penalização de 1/3 da cotação da pergunta.**
- 3. Devem ser entregues todas as folhas do exame.**

1 – Qual das seguintes afirmações descreve melhor a utilidade de existirem modos de processamento distintos (*user/kernel*)?

- a) Permitir que as aplicações experimentem instruções privilegiadas para otimizar desempenho
- b) Impedir que as aplicações em *user space* acessem directamente aos recursos do sistema, garantindo protecção e isolamento
- c) Evitar qualquer mudança de contexto durante operações de I/O
- d) Forçar todas as operações de I/O a serem realizadas em *user space* para facilitar o desenvolvimento das aplicações

2 – Em Linux, após a invocação da system call *fork()*, qual das seguintes afirmações sobre o espaço de endereçamento do processo filho é verdadeira?

- a) O processo filho partilha fisicamente toda a memória com o processo pai
- b) O espaço de endereçamento do filho é completamente novo, sem qualquer relação com o do processo pai
- c) O processo filho tem um espaço de endereçamento distinto, mas mapeado para as mesmas páginas físicas, sendo separada apenas após uma operação de escrita
- d) O processo filho utiliza exclusivamente a memória do *kernel* até invocar uma função *exec*

3 – Num sistema com escalonamento preemptivo, que papel desempenha o temporizador de hardware (*hardware timer*)?

- a) O temporizador é apenas informativo, uma vez que a preempção ocorre quando o processo em execução liberta voluntariamente o CPU
- b) O temporizador gera interrupções periódicas que permitem ao *kernel* interromper o processo corrente e escolher outro (*timer interrupt*)
- c) O temporizador executa, por si só, o código do escalonador em *user space* para decidir o processo que irá executar a seguir
- d) O temporizador sinaliza o processo em execução para invocar a função *yield()*

4 – Nos *kernels* modulares (por exemplo, Linux com módulos carregáveis), qual a principal finalidade dos módulos?

- a) Executar todo o código do sistema em *user space*
- b) Permitir adicionar/atualizar funcionalidades (ex., *device drivers*) sem recompilar ou reiniciar o *kernel*
- c) Forçar o escalonador a usar políticas diferentes por processo
- d) Garantir que o *kernel* nunca faz I/O diretamente

5 – Porque é que um processo pode ficar no estado "zombie" (*defunct*) após terminar?

- a) Porque está à espera de mais tempo de CPU para executar operações de libertação de recursos
- b) Porque o *kernel* mantém o registo do PID e do estado de saída até o processo pai invocar `wait()` / `waitpid()` para recolher esse estado
- c) Porque o processo nunca invocou `exit()` explicitamente
- d) Porque o processo está bloqueado numa operação de I/O e não pode terminar

6 – No problema Produtor-Consumidor com um *buffer* circular limitado a N posições, que conjunto de primitivas é mínimo e suficiente para coordenar múltiplos produtores e múltiplos consumidores sem condições de corrida?

- a) Um semáforo de exclusão mútua que protege o *buffer*
- b) Dois semáforos de contagem, um para o número de *slots* livres e outro para o número de itens no *buffer*
- c) Um semáforo de contagem para *slots*, outro para itens, mais um semáforo de exclusão mútua que protege o *buffer*
- d) Somente operações atômicas de leitura/escrita sem semáforos

7 – No contexto do problema dos Leitores-Escritores, qual é a consequência possível se existir um fluxo contínuo de leitores na primeira variante da solução que favorece os leitores?

- a) Os leitores podem experienciar inanição (*starvation*) e, por isso, não progridem
- b) Os escritores podem experienciar inanição (*starvation*) e, por isso, não progridem
- c) Nem leitores nem escritores podem aceder à zona de memória partilhada
- d) O sistema fica obrigado a usar semáforos binários em modo reentrante

8 – Quando é apropriado usar um `pthread_rwlock_t` em vez de um *mutex* tradicional?

- a) Quando a aplicação tem muito mais escritores do que leitores e quer reduzir o custo (*overhead*) do esquema de sincronização
- b) Quando é necessário permitir múltiplos leitores simultâneos, mantendo exclusão para escritores, nomeadamente em cenários com muitos leitores e poucos escritores
- c) Quando se quer garantir que escritores nunca ficam bloqueados
- d) Quando se pretende evitar qualquer possibilidade de inanição (*starvation*) para leitores e escritores sem configuração adicional

9 – No contexto do escalonamento por filas multinível com feedback, qual é o principal benefício de permitir que os processos se movam entre as diferentes filas?

- a) Simplificar a implementação do escalonador, eliminando a necessidade de prioridades fixas
- b) Garantir que todos os processos recebam um *time slice* idêntico, independentemente do seu comportamento
- c) Adaptar dinamicamente o escalonamento ao comportamento dos processos (ex., I/O-bound vs. CPU-bound) e mitigar a inanição (*starvation*) através de mecanismos de promoção
- d) Reduzir o custo (*overhead*) da comutação de contexto, pois os processos permanecem na mesma fila até à sua conclusão

10 – Considere um sistema operativo que utiliza escalonamento *Round-Robin* (RR). Qual das seguintes afirmações sobre o algoritmo RR e o seu *time slice* é correta?

- a) Um *time slice* muito longo no RR resulta numa melhor performance interativa, pois os processos têm mais tempo para completar as suas tarefas
- b) O RR é um algoritmo não-preemptivo que garante que os processos *CPU-bound* não monopolizem a CPU
- c) Se o *time slice* for excessivamente curto, o débito computacional (*throughput*) do sistema pode sofrer significativamente devido ao aumento do *overhead* da comutação de contexto
- d) O RR elimina completamente o *overhead* da comutação de contexto, tornando-o o algoritmo mais eficiente em todas as situações

Grupo II (12/20 valores)

Versão A

NOTAS:

1. Responder cada questão numa folha separada, devidamente identificada.
2. Indicar a versão do exame em cada uma das folhas.
3. Não é necessário indicar o nome das bibliotecas usadas na resolução.

[5 valores] 1) Uma empresa de retalho tem os dados de vendas do ano anterior registados num *array sales*, e pretende identificar os registos onde foram vendidos mais de 2000 artigos. Como a empresa tem 1 000 000 registos, quer otimizar esta tarefa usando 10 processos, cada um responsável por procurar em 100 000 registos. Considera que cada registo é uma estrutura com os campos *customer_code*, *product_code*, *quantity*.

Sempre que um processo filho encontrar um produto que satisfaça o critério de pesquisa, deve enviar o código do produto para o processo pai, que o armazena num *array larger*, imprimindo o seu conteúdo quando todos os filhos tiverem terminado. Assuma que os valores do *array sales* já estão preenchidos com valores aleatórios.

A sua solução deve garantir a criação do número adequado de processos e de *pipes*, permitindo que **os processos filho sejam executados de forma concorrente**.

[7 valores] 2) Implemente um programa em C que simule um sistema de monitorização ambiental com **3 sensores** (*threads* produtoras) e **2 threads consumidoras** (registo e alerta).

As *threads* comunicam através de um **buffer circular** com capacidade para **10 leituras** (cada leitura é uma estrutura com os campos *sensor_id*, *read_number*, *value*, *urgent_flag*). Cada sensor gera **40** leituras incrementais (ex.: 15, 16, 17, ...), coloca-as no *buffer* circular e ativa a *urgent_flag* sempre que o valor ultrapassa um limiar definido (ex.: 30).

A *thread* registo lê cada leitura do *buffer* e imprime algo como:

Leitura 3 do Sensor A: 17

Leitura 2 do Sensor B: 23

...

Garanta que:

- Se o *buffer* estiver cheio, as *threads* sensores ficam **bloqueadas de forma passiva** até haver espaço livre;
- Qualquer acesso ao *buffer* (leitura ou escrita) ocorre sob **exclusão mútua**;
- Se o *buffer* estiver vazio, a *thread* registo fica **bloqueada de forma passiva** até surgir nova leitura;
- Leituras do mesmo *sensor_id* devem ser processadas na mesma ordem em que foram geradas (*read_number* crescente);
- A *thread* alerta mantém um contador de urgências por sensor e imprime uma mensagem quando é detetada uma *urgent_flag* (ex.: ALERTA 10, Sensor B: leitura 12, valor 33);
- A *thread* alerta não deve impedir a *thread* registo de processar leituras. Deve ser acordada através de um mecanismo de notificação adicional.